

Exploring the effectiveness of service migration strategies for virtual network embedding

Federico Giarre, Yassine Hadjadj-Aoul*

Univ Rennes, Inria, CNRS, IRISA, Rennes, France

ARTICLE INFO

Keywords:

Network slicing
Virtual network embedding
Service migration
Reinforcement learning
Network robustness

ABSTRACT

Network slicing, a key component of post-5G networks, has brought virtual network embedding (VNE) to the forefront of networking research. However, existing VNE approaches are limited by their consideration of static network topologies, failing to account for the dynamic nature of real-world networks. This limitation becomes particularly problematic in the face of link failures or topology modifications, rendering these approaches ineffective. To address this, we propose a new service placement strategy that remains effective even when the network topology changes. Furthermore, we introduce several service migration strategies and thoroughly investigate their effectiveness. Our results demonstrate the adaptability of our proposed strategy, which leverages graph neural networks, in handling link failures without necessitating relearning. This adaptability underlines the potential of our approach to significantly enhance the robustness and flexibility of service migration in VNE, thereby contributing to the evolution of network slicing in post-5G networks.

1. Introduction

In the rapidly evolving telecommunications sector, network slicing is emerging as a cornerstone of post-5G networks [1]. This technology, which allows for the creation of multiple virtual networks on a single physical infrastructure, is revolutionizing the way we manage and operate networks. However, realizing this vision depends on our ability to effectively enforce Virtual Network Embedding (VNE), which is a crucial yet complex aspect of network slicing [2].

Traditional approaches to network slicing often consider static network topologies, an increasingly inadequate perspective in today's rapidly changing network environments [3]. Link failures, topology modifications, and other unexpected changes can make these static methods ineffective, highlighting the need for a more robust network capable of migrating services to adapt to the ever-changing networks.

The literature on service migration has evolved significantly over the years. While classical research has focused on single Network Functions (NFs) migration or paths' replacements, recent studies have highlighted the importance of considering graph-structured services. Various methodologies, ranging from Integer Linear Programming (ILP) to heuristics and metaheuristics, have been proposed to address the challenges of service migration in dynamic networks [4]. However, many of these approaches are limited in handling complex network scenarios, particularly those involving changing topologies. Our work builds on these foundations, aiming to address the identified gaps and provide a more comprehensive solution for service migration in post-5G networks.

In this paper, we tackle the challenges above. We propose and analyze several service migration strategies capable of efficiently placing services even during topology changes. Our approach leverages the power of Graph Neural Networks (GNN), allowing for a high degree of adaptability without the need for relearning [5]. These contributions represent a significant advancement in the field of VNE, which traditionally considers only static topologies, offering new avenues for enhancing the robustness and flexibility of network slicing in post-5G networks.

The remainder of this paper is organized as follows: In Section 3 we delve into the VNE problem, providing a comprehensive formalization and highlighting the methodology employed for its resolution. Section 2 introduces a brief overview of the related work and positions our contribution. Section 4 focuses on the Virtual Network Request (VNR) migration problem, where we introduce and formalize the concept. In Section 5, we detail the migration strategies we propose, elucidating their design and functionality. In Section 6, we present a detailed evaluation of our strategies, showing their advantages and limits. Finally, Section 7 concludes the paper and discusses potential future research.

2. Related work

The advent of network slicing in 5G networks has reinforced the importance of service migration strategies in future networks, particularly as these networks become increasingly complex and dynamic. In

* Corresponding author.

E-mail addresses: federico.giarre@inria.fr (F. Giarre), yhadjadj@irisa.fr (Y. Hadjadj-Aoul).

fact, service migration plays a crucial role in maintaining network performance and reliability, particularly in scenarios of changing network conditions (e.g., network congestion, link degradation or failure) [4]. In such situations, the ability to quickly and seamlessly migrate services to less congested or more reliable parts of the network can significantly improve network performance and user experience [6].

Existing work can be classified according to several criteria: the **triggers for migration**, which could include congestion detection or avoidance, network defects, or energy consumption optimization; the **methodology** employed, which could be either reactive or proactive; and the **characteristics of the services** being migrated, whether they are monolithic or graph-structured; and the **resources' allocation/reallocation** algorithms. The different points listed here are orthogonal and can be present together [7], but in what follows we will only focus on the last two points.

2.1. Services' characteristics

The characteristics of the services being migrated can significantly influence the migration strategy. Monolithic services, which are self-contained and independent, can often be migrated more easily than graph-structured services, which consist of multiple interconnected components. Indeed, the migration of graph-structured services requires a guaranteed networking resources to maintain the dependencies and interactions between different components. The majority of literature falls into the first category, i.e., where we have monolithic services [4].

Furthermore, services can be either constrained or unconstrained. Constrained services, such as those explored in this study, have specific requirements. These may include the amount of bandwidth that must be available between the services' components, particular latency concerns, and other system requirements. This complexity of requirements greatly complicates the resource allocation process and, in particular, the migration process.

2.2. Resource allocation/reallocation algorithms

Resource allocation/reallocation algorithms are crucial in service migration, and the complexity of the algorithms involved often depends on the available resources and the constraints of the services to be migrated [8]. Indeed, in the case of over-dimensioned systems with 'best-effort' monolithic services, a very basic migration strategy, such as First-Fit, might be quite sufficient [9]. The issue becomes significantly more complex when resources are not abundant (e.g., network edges) and when dealing with constrained services structured as graphs. Our approach falls into the latter category, in which we delve in the rest of the section.

The approaches proposed in the literature can be classified in different ways (see the summary in Table 1).

While the literature contains limited studies specifically addressing the service migration problem, it is worth noting that traditional research on service placement could be adapted for redeploying services, just as new ones. However, migrating a service without considering its current placement can be inefficient, as demonstrated in this paper, and might even risk destabilizing the network.

Most of the literature approaches formalize the problem using Integer Linear Programming (ILP) [10,12,14,16–19,23–25]. Given the NP-completeness of the problem, these methods invariably suggest alternative solutions for better scalability. An exact model is typically limited to solving small problem instances. Moreover, without knowledge of future requests, such models are not efficiently applicable and cannot guarantee optimality [27].

The prevalent heuristics to address the migration challenge often employ "First-Fit" or "Decreasing First-Fit" strategies for node placement [10–21], with variations to optimize energy consumption or

minimize congestion [11,12]. For virtual link placement, these strategies typically rely on shortest-path calculations. While effective for determining the shortest path between two points, they fall short when placing graphs. These heuristics often underperform. More sophisticated heuristics, like those based on the Hopcroft–Karp [21] or Edmonds–Karp algorithms [18], offer worst-case guarantees. While not optimal, they present more efficient alternatives.

Meta-heuristics, such as "Ant colony" [22], "Bee colony" [24], "Simulated Annealing" [25], and "Particle-swarm" [23], have been introduced to identify global optima for the resource relocation challenge. These methods have proven to be adapted to tackle complex combinatorial problems. They facilitate a comprehensive exploration of the solution space, but require a digital twin, a precise replica of reality, to test the solutions' efficiency before the placement. This requirement can be costly. In addition, each relocation request requires a new calculation, as this type of approach is deprived of any learning.

Recent advancements have applied reinforcement learning to the VNE problem, yet few have specifically tackled service migration. The Irregular Cellular Learning Automata (ICLA) method, primarily focusing on SDN controller and flows placements, has been suggested [26]. This method, however, presumes a limited state and action spaces for timely convergence, which is not the case for VNE. More critically, such techniques demand recurrent learning for convergence, rendering them impractical for real-world scenarios.

In our study, we aim to advance the current state of the art by introducing a technique that not only identifies superior solutions or even the global minimum using deep reinforcement learning but also eliminates the need for repetitive learning for each migration computation through the integration of Graph Neural Nets (GNN). In addition, we evaluate various migration strategies, demonstrating that approaches based on partial migration offer the best balance between cost and efficiency.

3. VNE mapping problem

3.1. Problem description

The Virtual Network Embedding (VNE) problem involves the allocation of physical resources to Virtual Network Requests (VNRs). A VNR is a graph that connects Virtual Network Functions (VNFs) with specific resource requirements. For the sake of simplicity, we will focus solely on the CPU requirements that the VNFs will demand from the physical nodes of the underlying topology (i.e., substrate network).

In this work, we will take into account a variation of the virtual node mapping (VNM) process where the mapping function, $f_{\text{VNM}}(\text{VNR}) : \mathbb{N}^{\text{VNR}} \rightarrow \mathbb{N}^s$ is injective VNR-wise. This implies that VNFs belonging to the same VNR cannot be placed on the same substrate node. Additionally, to connect different VNFs, virtual links (VLs) are mapped onto the topology using a virtual link mapping (VLM) function $f_{\text{VLM}}(\text{VNR}) : \mathbb{L}^{\text{VNR}} \rightarrow \mathbb{L}^s$. One VL is composed of the succession of substrate links that it has to traverse to get from one VNF to another (P_{lv}). Each VL requires a certain amount of bandwidth for its deployment, and it is possible for multiple VLs (even belonging to the same VNR) to share the same substrate link. The substrate link has a limited total bandwidth capacity, which serves as a constraint when deploying VLs onto it.

We can formalize the resource constraint for both f_{VNM} and f_{VLM} for the placement as:

$$c_{n^v} \leq c_{n^s}^i \quad \text{where} \quad c_{n^s}^i = c_{n^s}^{0,i} - \sum_{n \in \mathbb{N}_s^v} c_{n^v} \quad (1)$$

$$b_{lv} \leq b_{ls}^i \quad \text{where} \quad b_{ls}^i = b_{ls}^{0,i} - \sum_{l \in \mathbb{L}_s^v} b_{lv} \quad (2)$$

In Eq. (1), c_{n^v} represents the resource requirement of VNF n^v , and $c_{n^s}^i$ represents the available residual CPU resources at the placement location n^s after considering the resource consumption of all currently placed VNFs. This constraint guarantees that the placement of VNFs is

Table 1
Summary of existing approaches.

Ref.	Type of migration				Type of network	Type of algorithm	Exact	Objective	Triggering event	Re-learning
	NFs	Paths	Part S.	Whole S.						
[10]				✓	DC	Heuristic	ILP	SA	Any	Not applicable
[11]				✓	DC	Heuristic		Energy	Any	Not applicable
[12]				✓	DC	Heuristic	ILP	Energy	Any	Not applicable
[13]		✓			WAN	Heuristic		SA	Any	Not applicable
[14]				✓	WAN	Heuristic	ILP	R2C	Mobility	Not applicable
[15]				✓	WAN	Heuristic		SA	Any	Not applicable
[16]				✓	WAN	Heuristic	ILP	R2C	Mobility	Not applicable
[17]				✓	WAN	Heuristic	ILP	R2C	Any	Not applicable
[18]			✓		WAN	Heuristic	ILP	R2C	Failure	Not applicable
[19]			✓		WAN	Heuristic	ILP	R2C	Any	Not applicable
[20]			✓		WAN	Heuristic		R2C	Any	Not applicable
[21]			✓		WAN	Heuristic		R2C	Mobility	Not applicable
[22]	✓				DC	Meta-heuristic		Energy	Any	Needed
[23]		✓			WAN	Meta-heuristic	ILP	Energy	Any	Needed
[24]				✓	WAN	Meta-heuristic	ILP	SA	Any	Needed
[25]				✓	WAN	Meta-heuristic	ILP	R2C	Congestion	Needed
[26]		✓			WAN	RL		R2C	Any	Needed
Our	✓	✓	✓	✓	WAN	DRL		R2C	Any	Not needed

Table 2

Notation.

Notation	Description
$\mathcal{G}^s, \mathcal{G}^v$	Substrate network and VNR graphs
$\mathbb{N}^s, \mathbb{N}^v$	Set of substrate nodes and VNFs
$\mathbb{L}^s, \mathbb{L}^v$	Set of substrate links and VNs
$\mathbb{N}^r, \mathbb{L}^r$	Set of VNFs and VNs to remap
$\mathbb{N}_i^s, \mathbb{L}_i^s$	Set of VNFs and VNs on node n^s or link l^s
$\mathbb{N}^v, \mathbb{L}^v$	Set of VNFs and VNs of a VNR V
$\mathcal{R}(v)$	Revenue to cost of the VNR v
$\mathcal{M}(\mathbb{N}^r, \mathbb{L}^r)$	Migration cost of a VNR
$c_{n^s}^{0,i}$	Overall CPU capacity of sub. node n^s at time i
$c_{n^s}^i$	Available CPU at sub. node n^s at time i
$b_{l^s}^{0,i}$	Bandwidth capacity of sub. link l^s at time i
$b_{l^s}^i$	Available Bandwidth at sub. link l^s at time i
$b_{l^s}^{i^v}$	Bandwidth requested by VNF n^v
c_{n^v}	CPU requested by VNF n^v
b_{l^v}	Bandwidth requested by VL l^v
P_v	Substrate path for a VL l^v
$ P_v $	Number of substrate link hops for a VL l^v

resource-aware, preventing the overloading of the substrate nodes and ensuring the efficient utilization of available resources in the network.

Eq. (2) applies similar reasoning to the bandwidth constraint. Here, b_{l^v} represents the resource requirement of VL l^v , and $b_{l^s}^i$ represents the available residual bandwidth resources on the physical link l^s . The constraint ensures that a VL can only be placed if there are sufficient residual bandwidth resources available after accounting for the existing VNs on that link (see Table 2).

The goal of the VNE mapping problem is to map a VNR graph with the best possible fit onto the substrate network. In this study, we focus solely on evaluating the quality of the placement based on the distance (measured in hops) between the VNFs. Thus, a perfect fit will have VNFs connected by VNs that are formed by only one substrate link. In order to measure this, we utilized the Revenue to Cost ($\mathcal{R}(\text{VNR})$) measure, which is computed as:

$$H_{l^v} = \begin{cases} \rho b_{l^v} (|P_v| - 1) & \text{if } |P_v| > 1 \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

$$\mathcal{R}(\text{VNR}) = \frac{\sum_{l^v \in \mathbb{L}^V} b_{l^v}}{\sum_{l^v \in \mathbb{L}^V} b_{l^v} + H_{l^v}} \quad (4)$$

The factor described in Eq. (3) is meant to be penalizing the $\mathcal{R}(\text{VNR})$ when paths P_v are composed of more than one substrate link. To avoid penalizing too much the Revenue to Cost of a VNR due to longer substrate hops, the discount factor ρ is applied. As for Eq. (4), the $\mathcal{R}(\text{VNR})$ will return a value in the range $[0, 1]$, where a higher value corresponds to a better placement.

3.2. DRL approach

The problem defined in the previous section is a more complex version of the knapsack problem, which is already NP-complete and thus very hard to solve with exact optimization techniques. Moreover, the system under consideration is dynamic (dynamic arrival and departure of services), which makes classical methods unsuitable [27]. Indeed, since we do not have access to future requests, we cannot guarantee optimality. However, learning techniques are able to generalize from past experiences and enable a real-time execution once trained.

As labeled data is not available, and knowing the superiority of reinforcement learning strategies [28], we decided to tackle the node mapping phase of the problem by proposing a Deep Reinforcement Learning (DRL) Agent.

In order to build such an agent, we need to express the VNE mapping problem as a Markov Decision Process [29] (MDP). In the following, we outline this MDP through its state representation, action space, and reward function (see Fig. 1).

3.2.0.1. State representation. The state is given by the pair $(\mathcal{G}_i^s, \mathcal{G}_i^v)$. Each substrate node at time i has two defining features: $c_{n^s}^{0,i}$ and $b_{l^s}^{0,i}$. VNFs are characterized by their CPU and bandwidth demands, represented as c_{n^v} and b_{l^v} .

3.2.0.2. Action space. The action space only concerns the VNM phase of the VNR placement. Let $\mathcal{A}_i$ represent the set of potential substrate nodes for deploying the i th VNF of the VNR, and a_i be the chosen action. Given that VNFs are placed in sequence and the VNM function's injectivity must be maintained, the action space can be expressed as:

$$\mathcal{A}_i = \mathcal{A}_0 / \{a_1 \dots a_{i-1}\}$$

3.2.0.3. Reward function. The reward function is designed to guide the Agent towards optimal VNR placements. It operates as follows:

- **Successful Placement:** If the Agent successfully completes both the VNM and VLM phases, it receives a reward equivalent to the R2C of the placed VNR. This encourages the Agent to find placements that optimize the R2C metric.
- **Unsuccessful Placement:** In cases where the Agent fails in either the VNM or VLM phases, a penalty of -1 is given. This discourages the Agent from making placements that lead to failures.
- **Intermediate Placements:** During the process, each VNF is placed sequentially. For these individual or intermediate placements, the Agent receives a neutral reward of 0. This ensures that the Agent does not receive premature positive or negative feedback before the entire VNR is placed.

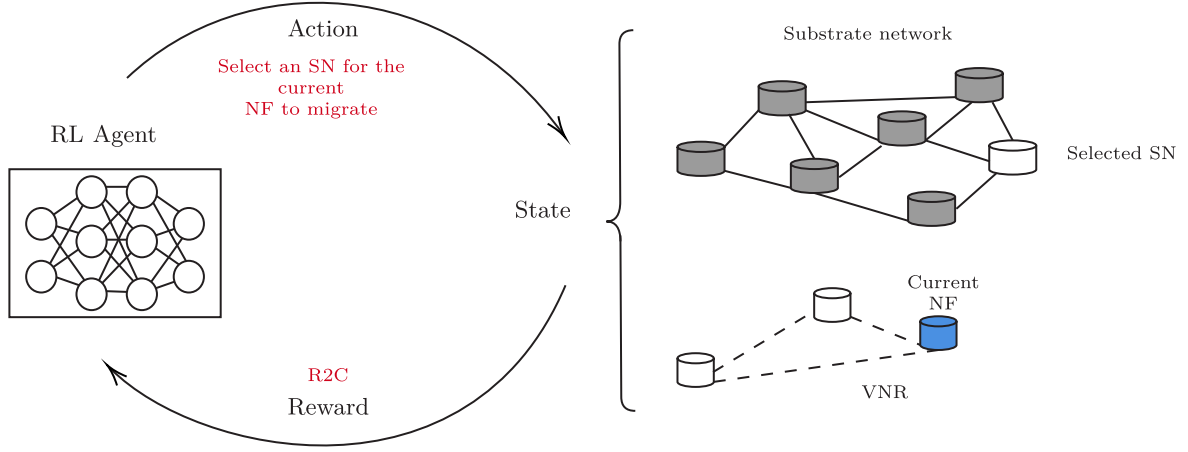


Fig. 1. MDP for VNR migration.

In order to prioritize immediate over distant rewards, the reward function incorporates a discount factor, denoted as γ . This not only accelerates decision-making but also aids in the learning process's convergence toward optimal outcomes.

The DRL agent we propose leverages a Graph Convolutional Network (GCN) [30] to extract key features from the substrate network, such as bandwidth and CPU availability in nodes and links, as well as details about the VNR to be mapped. Upon processing these features, the agent attempts to place the VNFs and receives the corresponding reward. After an initial exploration phase where the agent takes random actions, it begins to adopt policies that optimize the mapping process.

This paper builds on the previous work in [31], which introduced a novel GCN model for a batch of services placement. While we retain the models and specifications from the original study, our contribution broadens their application to accommodate dynamic service arrivals and departures, as well as network changes such as link failures and updates, which can necessitate service migrations.

4. VNR migration

VNFs within the same VNR need to cooperate in order to achieve a common objective. This implicitly requires Quality of Service (QoS) measures in case the physical links that host the connections of such VNFs degrade or break. We tackled this problem by proposing link-aware migration strategies to be applied to the VNR in case of link deterioration or outage.

In the following analysis, we focus solely on single-link deterioration/failures while ignoring multiple-link deterioration/failures for simplicity. In cases where dependent links fail, our approach consists of replacing the dependent links with a single link that combines them. This allows us to capture the interdependencies between the links while simplifying the analysis. This is a reasonable approximation, as the likelihood of concurrent failures among multiple independent links is typically small. In scenarios involving multiple link failures, our assumption of a Poisson arrival process for failures implies that the time interval between two events may be minimal, although not equal to zero. Therefore, our assumption of a single-link failure is a close approximation to reality.

4.1. Problem description

In our system, whenever a physical link experiences a failure or degradation, the migration process is promptly initiated. If the process is successful, meaning that the Agent was successful in finding a remapping for the affected resources, we proceed with committing the operation. However, if the Agent fails, the entire VNR will be extracted from the network and placed in a priority queue.

This queue serves as a temporary waiting area for VNRs that cannot be immediately accepted due to resource constraints or other factors. The priority queue is managed according to specified criteria, such as arrival time, resource requirements, and service priority. This ensures that VNRs are processed efficiently, minimizing any disruption of the network's overall performance.

Each time a resource in the network becomes available (i.e., when other VNRs leave the network), the Agent tries mapping queued VNRs according to their priorities. Popping a VNR from the queue can happen in three cases: (i) The Agent is able to map the VNR correctly; (ii) The Agent encounters repeated failures in mapping the VNR for a predefined number of attempts (k), where k can be configured to align with the system's requirements or the criticality of the service; (iii) The VNR exhausts its available time in the system, signifying the end of its active state, and it can be removed from the queue.

Arriving VNRs are also enqueued, ensuring that extracted VNRs waiting to be remapped receive higher priority. This strategy is proposed to prevent starvation issues, especially when dealing with a high arrival rate of VNRs.

In case of a physical link outage, migration is initiated for every VNR that utilizes the affected link. However, when faced with link degradation, the Agent adopts a different approach. It performs migration for the minimum number of VNRs required to alleviate the saturation on the physical link, effectively restoring it to a non-saturated state. This strategy ensures that resources are optimally utilized, minimizing disruption to the network while efficiently mitigating the impact of link degradation.

4.2. Problem formulation

We can present link degradation or outage using the following expression:

$$b_{ls}^{0,i} < b_{ls}^{0,i-1} \quad \wedge \quad b_{ls}^i < 0 \quad (5)$$

In this formulation, $b_{ls}^{0,i}$ represents the bandwidth capacity of the physical link l^s at time i , and $b_{ls}^{0,i-1}$ denotes the bandwidth capacity on the same link at the previous time step. The inequality $b_{ls}^{0,i} < b_{ls}^{0,i-1}$ indicates link degradation. Additionally, the right-hand side of the equation means that the available bandwidth on the link is less than zero, reflecting the saturation state of the link, and therefore the need for migration.

Whenever a migration is triggered, we compute the migration cost of all involved VNRs using the expression:

$$\mathcal{M}(\mathbb{N}^r, \mathbb{L}^r) = \omega \sum_{n^v \in \mathbb{N}^r} c_{n^v} + (1 - \omega) \sum_{l^v \in \mathbb{L}^r} b_{l^v} \quad (6)$$

where ω represents a hyperparameter applied to the costs of migrating nodes and links in order to modify the importance of one with respect to the other.

Each VNR is enqueued in a priority queue, with its migration cost serving as the priority. Lower migration cost values are given higher priority, ensuring that VNRs requiring fewer resources to be migrated are placed before others in the queue.

The success of migration of a VNR V is contingent upon remapping all its nodes $n \in \mathbb{N}^r \cap \mathbb{N}^V$ and its links $l \in \mathbb{L}^r \cap \mathbb{L}^V$ without encountering any issues. This priority-based approach allows for efficient migration of VNRs with minimal resource disruption and optimizes resource utilization during the remapping process.

Considering that μ denotes the average value of $\mathcal{M}(\mathbb{N}^r, \mathbb{L}^r)$, we introduce a constant c such that $c \gg \mu$. When the migration process is unsuccessful, the corresponding Virtual Network Request (VNR) is enqueued with its priority modified to the following:

$$-c + \mathcal{M}(\mathbb{N}^r, \mathbb{L}^r) \quad (7)$$

This modification ensures that VNRs extracted from the network and awaiting remapping are assigned higher priority compared to arriving VNRs, while still maintaining the relative priority order between them. By employing this approach, we ensure the utilization of network resources while enhancing the overall service provisioning efficiency.

5. Migration strategies

In this section, we propose several migration strategies. These strategies vary in the selection of resources to be migrated, and hence the migration cost, ranging from the most DRL-intensive approach (whole VNR migration) to not utilizing DRL at all (edge remapping). Our study explores the effectiveness and trade-offs of these strategies, delving into their impact on network performance and resource optimization.

5.1. Whole VNR migration

This strategy involves migrating all nodes and edges of a VNR. When coming to link degradation, not only the directly connected VNFs are affected, but the entire service graph can experience degradation. Nonetheless this method incurs the highest migration cost among the three strategies.

For this strategy, the links and the nodes to remap are trivially computed as:

$$\mathbb{N}^r = \mathbb{N}^V$$

$$\mathbb{L}^r = \mathbb{L}^V$$

By migrating the entire VNR we address potential service interruptions and enable the DRL Agent to adapt and optimize its decisions over time. This iterative learning process enhances the strategy's efficiency and ultimately leads to improved resource utilization and better overall network performance.

Note that this method can be especially beneficial in situations in which the network is congested in a certain area, so migrating the entire VNR to a new location can be a good strategy.

5.2. VNR subgraph migration

This strategy consists in migrating all the VNFs attached to the ends of the VLs mapped on the degraded substrate link, and then remapping the virtual links to reconnect the VNFs. This allows the DRL Agent to find a placement only for the resources that would be impacted directly by the substrate link degradation, leading the Agent to have fewer resources to move with respect to migrating the whole VNR. For this strategy the links and the nodes to remap are computed as follows:

$$\mathbb{N}^r = \text{VNFs connected by } l^V \forall l^V \in \mathbb{L}_s^V$$

Table 3

Parameters used for running tests.

Parameter	Value
Simulation time	180000 ticks
Episode duration	300 ticks
Mean time between VNR arrival	5 ticks
Mean time between bandwidth variation	10 ticks

$$\mathbb{L}^r = \text{VLs attached to } n^V \forall n^V \in \mathbb{N}^r$$

This method will be beneficial in scenarios in which resources are available in the surroundings of the VNR, so not only the placement can have a similar Revenue to Cost, but the latter can also improve based on newly freed resources.

5.3. Edge remapping

This strategy involves a straightforward remapping of the VLs that were originally mapped on the degraded substrate link using Dijkstra's shortest path algorithm.

It does not involve the DRL Agent at all, since no remapping of nodes is required. Therefore, the results depend only on the network load. In this case, the cost of migration will be solely made of the number of virtual links that have to be remapped. With this strategy, we only need to compute the set $\mathbb{L}^r$, which consists of the VLs that were using the broken or degraded substrate link s .

We can finally compute the remapping set as:

$$\mathbb{L}^r = \mathbb{L}_s^V$$

By adopting this approach, we address links' failures or degradation efficiently and optimize network performance based on the network's current state without the need for complex DRL-based decisions.

6. Performance evaluation

In this section, the evaluation of the three strategies will be presented. Additionally, we will also address the migration problem using a First-Fit (FF) and a Decreasing First-Fit (DFF) agent. These agents serve as the basis for many strategies referenced in related work and will be used for comparative purposes [10–21].

6.1. Evaluation setup

The performance of the proposed strategies is carried out using a customized discrete-event simulator based on [32]. This simulator is specially designed to reproduce real network scenarios and dynamics, allowing us to comprehensively evaluate the performance of our strategies in a controlled environment [33].

To model the stochastic events (i.e., VNR arrivals and link failures or capacity modification), we considered a Poisson process, which is a common and effective method in network simulation studies to account for the randomness and unpredictability of these events [34]. For VNR departures, we adopt an exponential service time model, which is widely accepted to represent the varying lifetimes of VNRs in the network.

We defined the Mean Time Between Arrivals (MTBA) for each event in discrete simulation time, along with other relevant parameters listed in Table 3.

For testing purposes, during each bandwidth variation event, one used substrate link will be intentionally broken to trigger migration. Once the migration is resolved, the link is restored. The simulated

VNR will be placed in a network topology, specifically the BtEurope topology.¹

It is important to note that while we have presented results for a specific topology, the proposed approach can be applied to both smaller and larger network topologies. However, the convergence of the algorithm may vary depending on the topologies' structures. The BtEurope topology is a well-connected graph, with few standalone node chains. These topology characteristics enable the agent to learn how to map the nodes on the more connected substrate nodes, achieving a higher R2C. Different topologies structures highly influence the R2C, trivial example of this phenomenon happening are topologies such as "GlobalCenter", fully connected so that the R2C is close to a constant 1 from the very first epoch, effectively nullifying the learning or the topology "Renater 2010", composed few loops and mostly chains, resulting in a low R2C (and thus, reward) even after the full training. However, when it comes to size alone and comparable structures, the incorporation of GNNs enables generalization in the learning process, facilitating the adaptation to topologies of different sizes without requiring relearning [35]. While this aspect will be a focus of future work, our current approach demonstrates the adaptability of the learning process with small variations in the network topology (i.e., link failures).

6.2. Results

As explained in previous sections, the revenue-to-cost ratio (R2C for short) is a measure of the quality of placement, where a high R2C indicates that the DRL agent has been able to efficiently place the VNR graph onto the substrate network.

To measure the quality of a migration, we introduce the $\Delta R2C$ measure, which quantifies the percentage of improvement of the R2C after the resources' migration.

Let $R(V)$ be the R2C of a given VNR V before migration, and $R(V')$ be the R2C of the same VNR after the migration. The $\Delta R2C$ is then computed using the formula:

$$\Delta R2C = \frac{R(V') - R(V)}{R(V)} \quad (8)$$

This measure provides valuable insights into the effectiveness of the migration process conducted by the DRL agent. A positive value for $\Delta R2C$ indicates an improvement in revenue compared to the initial placement. Conversely, a negative value does not imply the agent's ineffectiveness, but rather reflects the challenge of remapping the VNR considering resource alterations.

The upcoming plots will follow a consistent format: each point on the graph represents the results of an episode, with an average of 30 migrations occurring within each episode. For each episode, the median of all $\Delta R2C$ s and the mean of all R2Cs will be displayed. A lighter area around the $\Delta R2C$ show the standard deviation of the migrations' $\Delta R2C$ of the episodes ($\sigma_{\Delta R2C}$), that will be both added and subtracted from the $\Delta R2C$ in order to indicate where the majority of results fall for an episode.

In the first three result plots (Figs. 4, 3 and 2), the mean R2C for the FF and DFF methods are shown, but not the $\Delta R2C$ and $\sigma_{\Delta R2C}$. This has been done in order to enhance the readability of such plots, yet the essence of the results can be gathered in the plots presented, later on, in Figs. 5 and 6.

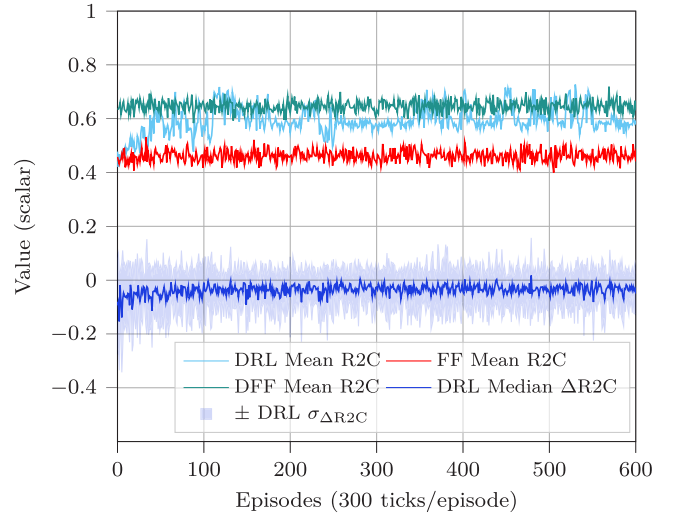


Fig. 2. Results for Edges' Remapping.

6.2.1. VNR's edges remapping

Fig. 2 illustrates the results for the edges' remapping strategy. In this case, the $\Delta R2C$ s systematically fall below the 0 line, indicating that this strategy leads to a decline in R2C. Indeed, the R2C of the migrated resources is found to be approximately 3% to 5% worse than the resources before migration. As this approach does not involve the use of the DRL agent during migration, the DRL Mean R2C measure does not influence the $\Delta R2C$ s, but it can be used to understand that the behavior of this strategy is not at all impacted by the initial placement of the VNRs by the controller.

Regarding $\sigma_{\Delta R2C}$, a linear trend is observed, with stability observed around 7.5% to 8%. This consistency in the standard deviation reflects the predictable performance of the edges' remapping strategy across multiple episodes.

6.2.2. VNR's subgraph migration

In Fig. 3, the results for the subchain migration strategy are presented. With this strategy, the $\Delta R2C$ s tends to be closer 0, indicating an equal (or eventually very close to equal) mapping for most episodes. Thus, by moving just a few more resources than the previous strategy, the DRL agent is able to more or less maintain the quality of the initial service placement, although without really being able to improve it significantly.

Since more resources are moved with respect to the previous strategy, a wider $\sigma_{\Delta R2C}$ is achieved, spanning between 7% to 8% of improvement/loss in R2C after the initial learning period.

6.2.3. Whole VNR migration

In Figs. 4 and 5 the results are shown for the Whole VNR Migration strategy. As it is possible to notice by the figures, the $\Delta R2C$ s is positive in several episodes, indicating that migration of the whole nodes and edges can result in a better (even if slightly) placement overall. The area given by the $\sigma_{\Delta R2C}$ also shows how the possible improvement (and deterioration) is wide enough to allow a 10% to 12% gain (or loss) in R2C when coming to migrations after the initial learning period.

6.2.4. First-Fit agent

The First-Fit agent have consistent results in all the executed tests: (i) The $\sigma_{\Delta R2C}$ in all tests is around 20% though not plotted for readability reasons. While not readable directly, this can be noticed from Figs. 5 and 6 where, excluding the outliers, the spread of the median $\Delta R2C$ is also in the order of 20% (ii) Even if the results of plots 5 and 6 show a good margin of improvement in all the tests (apart from the Edges remapping), due to the low R2C of the mapped VNRs it is easy to improve or worsen a placement sensibly.

¹ BtEurope and other topologies sources: <http://www.topology-zoo.org/explore.html>

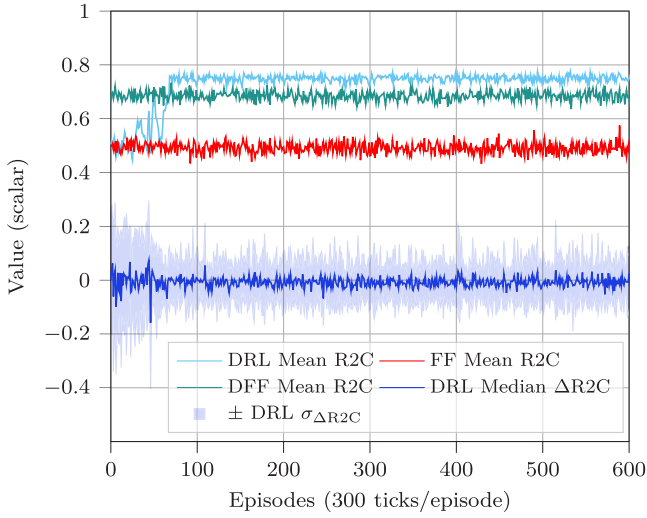


Fig. 3. Results for Subgraph Migration.

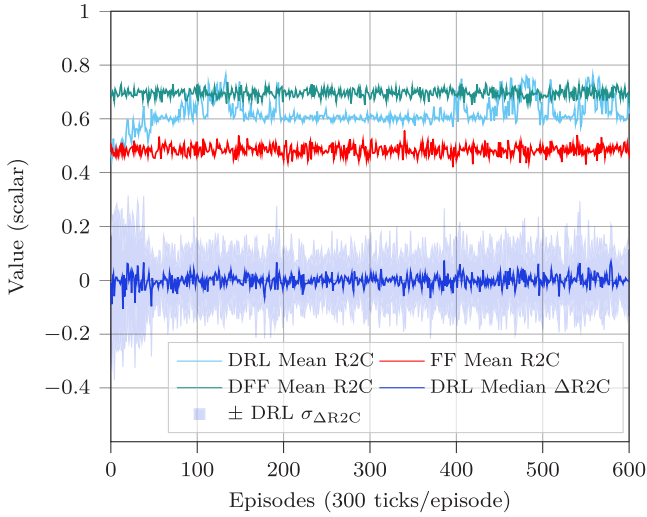
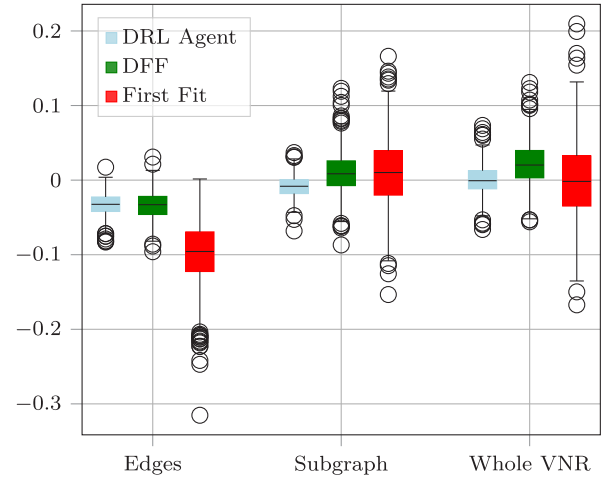
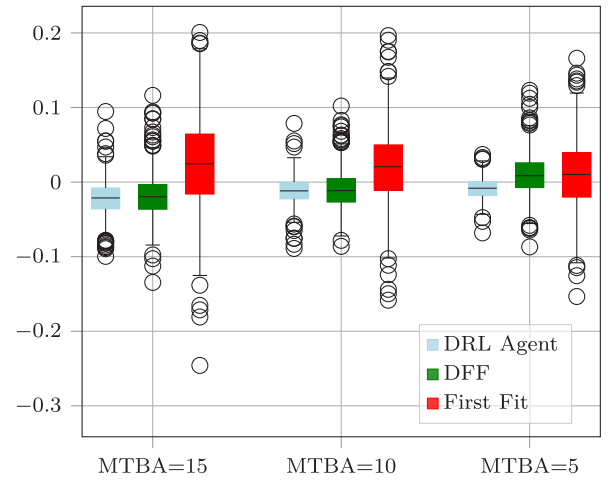


Fig. 4. Results for Whole VNR Migration.

6.2.5. Decreasing first fit implementation

Decreasing First-Fit (DFF) is a slight modification of the widely recognized First-Fit method. The difference from the classic First-Fit implementation is that there must be some kind of ordering for the containers to fill based on some property. In our implementation, since the R2C is related only to the length difference of VL and substrate paths, we want to ideally map the VNFs to substrate nodes that are well connected to the rest of the topology. This approach reduces the number of multi-hop VLs and subsequently lowers the penalty in the R2C calculation. To achieve this result, available substrate nodes are selected by sorting their degrees (number of connections to other nodes) in descending order. Examples of DFF implementation for node mapping similar to the one presented in this work can be found in the articles [10,18], while in a more general sense, other articles define a utility function thanks to which they are able to perform an ordering of the substrate nodes. As for the results, DFF achieves good Mean R2C from the very start, surpassing the results of the DRL Agent in the Whole VNR and Edge migration strategies. DFF is capable of performing good placements of a Whole VNR, thus achieving good R2C when VNFs do not move or are replaced completely. However, this higher R2C is achieved at the cost of a much wider spread of $\Delta R2C$ as seen in Fig. 5,

Fig. 5. Spread of the median $\Delta R2C$ values of the different strategies.Fig. 6. Spread of the median $\Delta R2C$ with subgraph strategy, different MTBA.

making it easier to improve or degrade the VNR sensibly when the migration of VNFs is needed.

6.3. Performance of the subgraph-based strategy under varied network loads

The results of the subgraph migration strategy offer the best trade-off in terms of cost-gain (and reliability) ratio, we decided to analyze the performance of it in less stressed environment. We conducted two more tests, where the only difference from the parameter in Table 3 is the mean time between arrival (MTBA) of new placement requests. While this does not directly affect the remapping of the VNR, there are some obvious consequences of making these changes. For instance, a less congested network allows for virtually better placements, but at the same time the DRL Agent will not learn what to do when more resources than usual are utilized. This factor leads to some good, yet counter-intuitive, results.

For the following tests, the mean time between arrival is set to 10 and 15 ticks. In Fig. 6, the results for these tests are plotted, accompanied by the result of the strategy with MTBA of 5 (as for the previous tests), for comparison.

To begin, let us consider the behavior of the First-Fit agent, which exhibits the expected natural response: when more resources are available, there are increased placement possibilities, leading to a wider range of potential improvement or degradation in the R2C metric. The

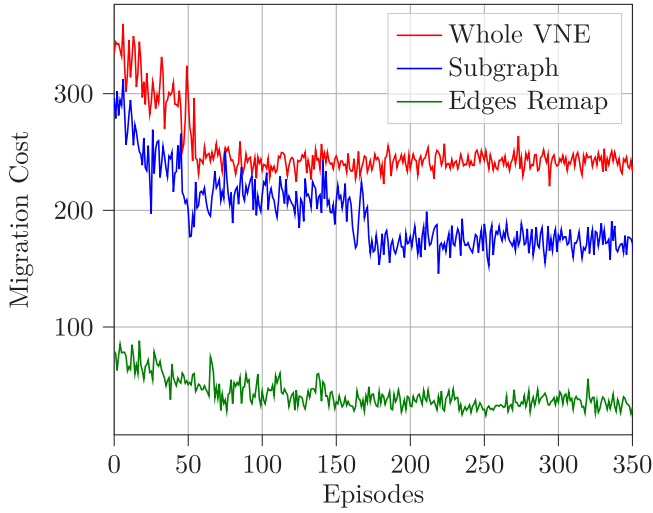


Fig. 7. Migration Cost for the different strategies.

same cannot be said about DFF, that achieve results similar to the DRL Agent, but with a much larger spread as the MTBA decreases.

For what concerns the DRL agent, the more loaded the network is, the more precise it gets with the placements, achieving a much narrower spread in the 5 ticks between arrivals compared to 10 or 15 ticks. This behavior arises from the agent's adaptation to the congested environment during interactions. In such situations, the agent learns to be mindful of other VNRs when migrating to avoid degrading the R2C. In contrast, in less congested scenarios, the agent may prioritize reaching higher R2Cs directly. Consequently, when the network is relatively loaded at migration time, the agent might be forced to take suboptimal actions, resulting in potential degradation in R2C.

Regardless of the test conditions, the subgraph strategy consistently maintains its unique properties. The observations discussed earlier remain valid, highlighting the strategy's stability and reliability. It consistently delivers enhanced R2C outcomes in various network scenarios.

6.4. Migration costs of different strategies

As explained in detail in Section 4, the cost of migrating a VNR (or more precisely, its elements to be migrated) is a measure used to guarantee the priority of remapping among extracted VNRs and new arrivals in the system. We can use this measure to compare the cost of using one strategy against the others.

In Fig. 7 the migration costs resulting from our tests are reported. After the first learning period, the DRL agent is able to take better decisions in placing the VNRs, decreasing the migration cost for all strategies. Excluding the Edge remapping strategy, which yields a low migration Cost by design, the trends for Whole VNR and Subgraph migrations are very similar and close for an initial period, then diverge more and more as the agent learns. This depends directly on how the Agent places the elements to be migrated in the subgraph strategy. Indeed, when more than one VL (from the migration elements) uses the same substrate link, the number of VNFs (and thus VLs) to be migrated is very close to the whole VNR. However, at around 150 episodes, the subgraph strategy Agent starts minimizing the overlap of VLs from the same VNR on identical substrate links, leading to consistently reduced migration costs. Investigating other functions to choose the migration items for the subgraph strategy could lead to even lower migration costs overall.

Table 4

Performance summary.

	Remapping strategy		
	Whole	Subgraph	Edge
Migration cost	High	Middle	Low
$\Delta R2C$	Better	Equal/Better	Worse
$\sigma_{\Delta R2C}$	$\pm 10\%$	$\pm 7\%$	$\pm 7\%$
R2C	Middle	High	Low

6.5. Remarks on the results

The results from the three strategies can be summed up in Table 4.

The results clearly indicate that no single strategy outperforms the others; each strategy comes with its own set of advantages and disadvantages. Nevertheless, certain conclusions can be drawn from the findings:

- Depending on the criticality of the service deployed and resources available, migrating the full graph leads to better results on average;
- If the service can account for little degradation, remapping the edges will be the fastest and the cheapest (in terms of the resources to be migrated) way of ensuring QoS;
- In a situation in which migration costs have to be minded, but service is critical, remapping a subgraph of the VNR will lead to equal or better results (on average) with a fair cost, while improving the overall R2C;
- When training the DRL Agent, a high load on the network can be beneficial in allowing the Agent to manage the opportunistic resources' allocation of other VNRs.
- While Decreasing First Fit can deliver R2C results comparable to the DRL ones (but without the computational complexity of the latter) it is also less precise and mindful of other events in the network, making it less suitable for service critical or efficient migrations of VNFs.

7. Conclusion

In this paper, we propose to extend a deep reinforcement learning model to solve service placement and migration problems. We have, thus, proposed several migration models that not only allow to react to congestion problems, but also to topological changes, due in particular to link failures. To this end, we have resorted to graph neural networks to avoid relearning each time a topological change occurs, unlike existing approaches. Among the strategies proposed, the strategy based on migrating the sub-graph directly affected by the network incident proved to be the best cost-performance trade-off, when compared with strategies that solely modify the links affected by the incident or those that migrate the entire service.

For the future work, our research agenda includes several directions. Firstly, we plan to delve deeper into the exploration of techniques for selecting specific sets of nodes and links for migration within the context of the sub-graph strategy. We aim to further refine our approach and enhance its adaptability to diverse network conditions. In addition, we recognize the need to study the impact of multiple link failures on network performance. Indeed, multiple simultaneous link failures present a significant challenge in network management and can severely disrupt service migration strategies. Therefore, we intend to investigate the effects of such failures and propose robust techniques to mitigate their impact.

CRedit authorship contribution statement

Federico Giarre: Conceptualization, Methodology, Validation, Writing – original draft, Writing – review & editing. **Yassine Hadjadj-Aoul:** Conceptualization, Supervision, Writing – original draft, Writing – review & editing.

Declaration of competing interest

Nothing to declare.

Data availability

I will share the data and source codes as soon as the paper is accepted.

Acknowledgments

This research work is conducted as part of the NaviDEC project, funded by the Brittany Region, France, under grant agreement #21004179.

References

- [1] Wen Wu, Conghao Zhou, Mushu Li, Huaqing Wu, Haibo Zhou, Ning Zhang, Xuemin Sherman Shen, Weihua Zhuang, AI-native network slicing for 6G networks, *IEEE Wirel. Commun.* 29 (1) (2022) 96–103.
- [2] Fadoua Debbabi, Rihab Jmal, Lamia Chaari Fourati, Adlen Ksentini, Algorithmics and modeling aspects of network slicing in 5G and beyonds network: Survey, *IEEE Access* 8 (2020) 162748–162762.
- [3] Yuan Yang, Mingwei Xu, Qi Li, Fast rerouting against multi-link failures without topology constraint, *IEEE/ACM Trans. Netw.* 26 (1) (2018) 384–397.
- [4] Shangguang Wang, Jinliang Xu, Ning Zhang, Yujiong Liu, A survey on service migration in mobile edge computing, *IEEE Access* 6 (2018) 23511–23528.
- [5] Jose Suarez-Varela, Paul Almasan, Miquel Ferriol-Galmes, Krzysztof Rusek, Fabien Geyer, Xiangliang Cheng, Xiang Shi, Shihan Xiao, Franco Scarselli, Albert Cabellos-Aparicio, Pere Barlet-Ros, Graph neural networks for communication networks: Context, use cases and opportunities, *IEEE Netw.* (2022) 1–8.
- [6] Zeineb Rejiba, Xavier Masip-Bruin, Eva Marín-Tordera, A survey on mobility-induced service migration in the fog, edge, and related computing paradigms, *ACM Comput. Surv.* 52 (5) (2019) 90:1–90:33.
- [7] Tarik Taleb, Adlen Ksentini, Pantelis A. Frangoudis, Follow-me cloud: When cloud services follow mobile users, *IEEE Trans. Cloud Comput.* 7 (2) (2019) 369–382.
- [8] Federico Giarrè, Yassine Hadjadj-Aoul, Soraya Ait-Chellouche, A service migration strategy for resilient multi-domain networks in outage scenarios, in: 2023 International Conference on Information and Communication Technologies for Disaster Management, ICT-DM, 2023, pp. 1–4.
- [9] Haotong Cao, Han Hu, Zhicheng Qu, Longxiang Yang, Heuristic solutions of virtual network embedding: A survey, *China Commun.* 15 (3) (2018) 186–219.
- [10] Nashid Shahriar, Reaz Ahmed, Aimal Khan, Shihabur Rahman Chowdhury, Raouf Boutaba, Jeebak Mitra, ReNoVatE: Recovery from node failure in virtual network embedding, in: 2016 12th International Conference on Network and Service Management, CNSM, 2016, pp. 19–27.
- [11] Xinjie Guan, Baek-Young Choi, Sejun Song, Energy efficient virtual network embedding for green data centers using data center topology and future migration, *Comput. Commun.* 69 (2015) 50–59.
- [12] Xinjie Guan, Baek-Young Choi, Sejun Song, Topology and migration-aware energy efficient virtual network embedding for green data centers, in: 2014 23rd International Conference on Computer Communication and Networks, ICCCN, 2014, pp. 1–8.
- [13] Mahboobeh Zangabadi, Christian Aguilar-Fuster, Javier Rubio-Loyola, A virtual network migration approach and analysis for enhanced online virtual network embedding, in: 2016 12th International Conference on Network and Service Management, CNSM, 2016, pp. 324–329.
- [14] Giorgos Chochlidakis, Vasilis Friderikos, Mobility aware virtual network embedding, in: 2015 IEEE International Conference on Communications, ICC, 2015, pp. 5846–5852.
- [15] Minlan Yu, Yung Yi, Jennifer Rexford, Mung Chiang, Rethinking virtual network embedding: substrate support for path splitting and migration, *ACM SIGCOMM Comput. Commun. Rev.* 38 (2) (2008) 17–29.
- [16] Yingying Guan, Yue Zong, Yejun Liu, Lei Guo, Zhaolong Ning, Joel J.P.C. Rodrigues, Virtual network embedding supporting user mobility in 5G metro/access networks, in: ICC 2019 - 2019 IEEE International Conference on Communications, ICC, 2019, pp. 1–7.
- [17] Sheng Zhang, Zhuzhong Qian, Jie Wu, Sanglu Lu, Leah Epstein, Virtual network embedding with opportunistic resource sharing, *IEEE Trans. Parallel Distrib. Syst.* 25 (3) (2014) 816–827.
- [18] Nashid Shahriar, Reaz Ahmed, Shihabur Rahman Chowdhury, Aimal Khan, Raouf Boutaba, Jeebak Mitra, Generalized recovery from node failure in virtual network embedding, *IEEE Trans. Netw. Serv. Manag.* 14 (2) (2017) 261–274.
- [19] Zhiping Cai, Fang Liu, Nong Xiao, Qiang Liu, Zhiying Wang, Virtual network embedding for evolving networks, in: 2010 IEEE Global Telecommunications Conference GLOBECOM 2010, 2010, pp. 1–5.
- [20] Ye Zhou, Yong Li, Depeng Jin, Li Su, Lieguang Zeng, A virtual network embedding scheme with two-stage node mapping based on physical resource migration, in: 2010 IEEE International Conference on Communication Systems, 2010, pp. 761–766.
- [21] Sherif Abdelwahab, Bechir Hamdaoui, Mohsen Guizani, Taieb Znati, Efficient virtual network embedding with backtrack avoidance for dynamic wireless networks, *IEEE Trans. Wireless Commun.* 15 (4) (2016) 2669–2683.
- [22] Xinjie Guan, Xili Wan, Baek-Young Choi, Sejun Song, Ant colony optimization based energy efficient virtual network embedding, in: 2015 IEEE 4th International Conference on Cloud Networking, CloudNet, 2015, pp. 273–278.
- [23] Sen Su, Zhongbao Zhang, Alex X. Liu, Xiang Cheng, Yiwen Wang, Xinchao Zhao, Energy-aware virtual network embedding, *IEEE/ACM Trans. Netw.* 22 (5) (2014) 1607–1620.
- [24] Zhu Qiang, Wanghui Qiang, Fengguang Sheng, LvHong Wu, Heuristic survivable virtual network embedding based on node migration and link remapping, in: 2014 IEEE 7th Joint International Information Technology and Artificial Intelligence Conference, 2014, pp. 181–185.
- [25] Shihabur Rahman Chowdhury, Reaz Ahmed, Nashid Shahriar, Aimal Khan, Raouf Boutaba, Jeebak Mitra, Liu Liu, ReVINE: Reallocation of virtual network embedding to eliminate substrate bottlenecks, in: 2017 IFIP/IEEE Symposium on Integrated Network and Service Management, IM, 2017, pp. 116–124.
- [26] Dipanwita Thakur, Manas Khatua, Multi-domain virtual network embedding with dynamic flow migration in software-defined networks, *J. Netw. Comput. Appl.* 162 (2020) 102639.
- [27] Pham Tran Anh Quang, Yassine Hadjadj Aoul, Abdelkader Outtagarts, A deep reinforcement learning approach for VNF forwarding graph embedding, *IEEE Trans. Netw. Serv. Manag.* 16 (4) (2019) 1318–1331.
- [28] Irwan Bello, Hieu Pham, Quoc V. Le, Mohammad Norouzi, Samy Bengio, Neural combinatorial optimization with reinforcement learning, in: 5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24–26, 2017, Workshop Track Proceedings, OpenReview.net, 2017.
- [29] Martijn Otterlo, Marco Wiering, Reinforcement learning and Markov decision processes, *Reinf. Learn.: State Art* (2012) 3–42.
- [30] Thomas N. Kipf, Max Welling, Semi-supervised classification with graph convolutional networks, 2017.
- [31] Anouar Rkhami, Tran Anh Quang Pham, Yassine Hadjadj-Aoul, Abdelkader Outtagarts, Gerardo Rubino, On the use of graph neural networks for virtual network embedding, in: 2020 International Symposium on Networks, Computers and Communications, ISNCC, 2020, pp. 1–6.
- [32] Meurer, et al., SymPy: Symbolic computing in Python, *PeerJ Comput. Sci.* 3 (2017) e103.
- [33] Ghina Dandachi, Sophie Cerf, Yassine Hadjadj Aoul, Abdelkader Outtagarts, Éric Rutten, A robust control-theory-based exploration strategy in deep reinforcement learning for virtual network embedding, *Comput. Netw.* 218 (2022) 109366.
- [34] Farah Slim, Fabrice Guillemin, Annie Gravey, Yassine Hadjadj Aoul, Towards a dynamic adaptive placement of virtual network functions under ONAP, in: 2017 IEEE Conference on Network Function Virtualization and Software Defined Networks, NFV-SDN 2017, Berlin, Germany, November 6–8, 2017, IEEE, 2017, pp. 210–215.
- [35] Benjamin Hudson, Qingbiao Li, Matthew Malencia, Amanda Prorok, Graph Neural Network Guided Local Search for the Traveling Salesperson Problem, 2021, ArXiv e-prints.



Federico Giarrè, Received the B.Sc. from the University of Trento, Italy in 2022. He is currently pursuing the EIT Digital's M.Sc. in Cloud and Network Infrastructure at both University of Rennes, France and KTH Royal Institute of Technology, Sweden. His current research interest areas include Cloud Infrastructures, Software Defined Networking and Network Functions Virtualization.



Yassine Hadjadj Aoul (yhadjadj@irisa.fr) is currently working as full professor at the University of Rennes, France, since 2022, where he is also a member of the IRISA Laboratory and the INRIA project-team Ermine. He received a B.Sc. in computer engineering with high honours from Mohamed Boudiaf University, Oran, Algeria, in 1999. Dr. Hadjadj received his Master's and Ph.D. degrees in computer science from the University of Versailles, France, in 2002 and 2007, respectively. He was an assistant professor at the University of Versailles from 2005 to 2007. He was also a post-doctoral fellow at the University of Lille 1 and a research fellow, under the EUFP6 EIF Marie Curie Action, at the National University of Dublin (UCD). Then, he joined the University of Rennes 1 as an associate professor. His main research interests concern the fields of wireless networking, multimedia streaming architectures and protocols, congestion control protocols and QoS provisioning, and satellite and space communications.